SDN + SOCKET PROGRAMMING · PROJECT 20

Adaptive Load Balancer

Six experiments: static round-robin vs adaptive, on Mininet with a Ryu controller

Leshawn Arron Lewis · Kishor Kumar M · Dhule Shashwati Vijay

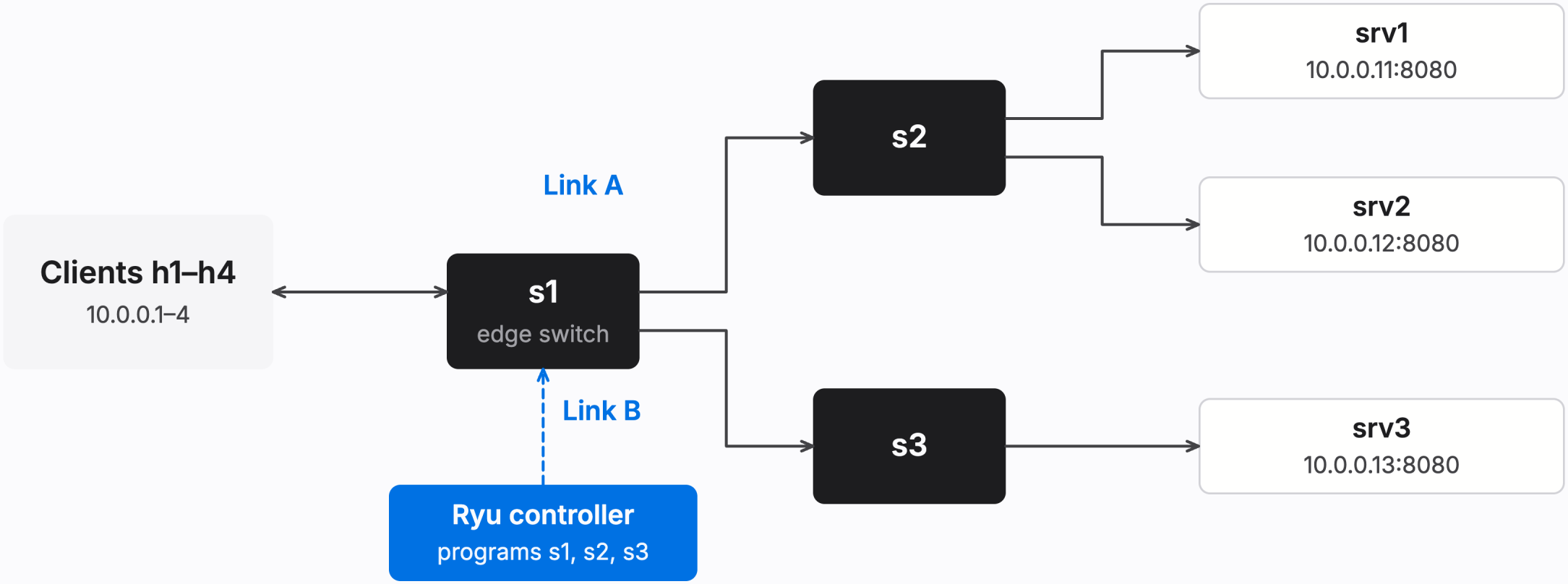
Team 9 · Section E · Jackfruit Mini Project

THE PROBLEM

Three servers, one address. Who gets the next connection?

Round-robin takes turns and never looks. Our controller checks each server's load and each link's traffic, and skips servers that are overloaded or dead.

The network: two paths, three servers



Clients connect to one virtual IP, 10.0.0.100. Links A and B are capped at 10 Mbit/s, and srv1 and srv2 share link A.

One new connection, step by step

1 Client sends a SYN to 10.0.0.100 →

2 s1 has no rule, so it asks Ryu →

3 Ryu scores each server and picks the lowest →

4 Ryu installs two rewrite rules on s1 →

5 The rest of the connection stays in the switch

MEANWHILE, EVERY 0.25 S

Each server sends a UDP load report to 10.0.0.254. No host owns that address, so a switch rule hands it to Ryu. This is where the socket app and the SDN meet.

How the controller decides

SCORE PER SERVER · LOWEST WINS

0.5 × load + 0.2 × CPU + 0.3 × link use

load = estimated connections ÷ capacity. The estimate adds flows sent since the last report and subtracts the ones the server has likely finished.

Healthy

Gets new connections when it has the lowest score.

Draining

At 90% of capacity: no new connections until it drops below 70%.

Down

No report for 3 s: its rules are deleted, clients reconnect elsewhere.

HOW WE TESTED

Six experiments, each run in both modes

#	Scenario	How we create it	Adaptive should
E1	Balanced	3 equal servers, steady clients	Match round-robin
E2	Unequal servers	srv3: 2 workers, runs 2× slower	Send srv3 fewer flows
E3	Hot server	h4 hits srv1 directly at t = 10 s	Drain srv1
E4	Server failure	srv2 process killed at t = 10 s	Stop using srv2 in 3 s
E5	Link congestion	9 Mbit/s iperf on link A at t = 10 s	Prefer srv3 on link B
E6	Scale-up	1, 2, 4, 8 threads per client host	Scale like round-robin

h1–h3 always generate the load. h4 is the disturbance in E3 and E5. One script, **run_all.sh**, runs all 18 runs and the analysis.

Each run: 30 s · 3 client hosts × 4 threads · static = round-robin baseline

What we measured

Throughput

Successful requests per second. Higher is better.

Mean latency

Average time for one request, in milliseconds.

p95 latency

95% of requests finish faster than this. It shows what the slowest users feel.

Failed requests

Share of requests that got an error or timed out.

Server share

The fraction of requests each server answered.

Jain's fairness

1.0 = every server equally busy for its size. 0.33 = one server does it all.

EXPERIMENT E1 · BALANCED LOAD

Equal servers: both modes perform the same

Throughput (requests/s), higher is better



p95 latency (ms), lower is better



Requests per server (srv1 / srv2 / srv3): static 33 / 33 / 33%, adaptive 33 / 32 / 35%.

RESULT

Same

within 1.5% throughput

With equal servers, round-robin is already the best choice. This is our sanity check: adaptive adds no overhead.

30 s per run · clients h1–h3 × 4 threads · full numbers in results/summary.csv

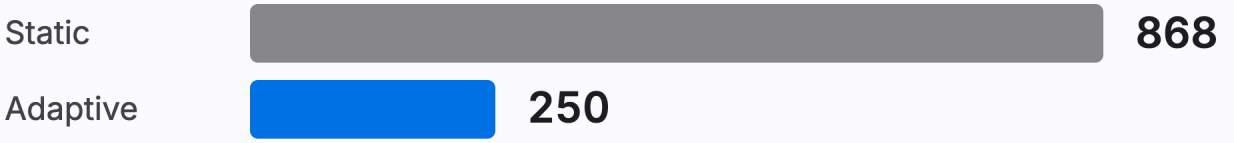
EXPERIMENT E2 · UNEQUAL SERVERS

A weak server: adaptive doubles throughput

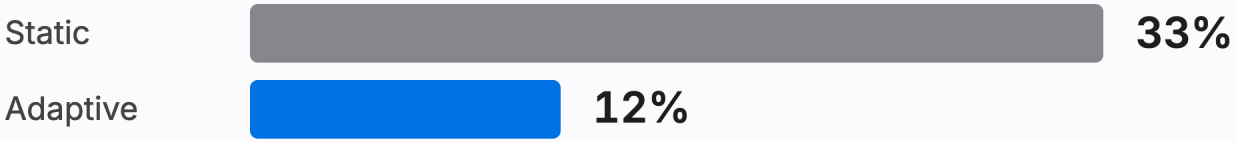
Throughput (requests/s), higher is better



p95 latency (ms), lower is better



Share of requests sent to weak srv3



RESULT

2×

the throughput

srv3 has 2 workers and runs 2× slower. Round-robin still sends it 1 in 3 requests, so they queue. Adaptive sees srv3's load and sends it only 12%.

30 s per run · clients h1–h3 × 4 threads · Jain's fairness 0.37 → 0.98

EXPERIMENT E3 · HOT SERVER

An overloaded server: adaptive drains it

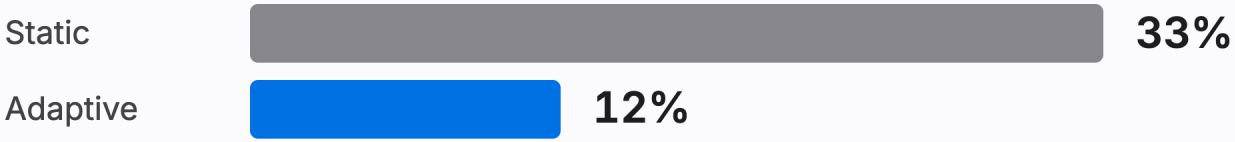
Throughput (requests/s), higher is better



p95 latency after t = 10 s (ms), lower is better



Share of balanced requests sent to hot srv1



RESULT

3.4×

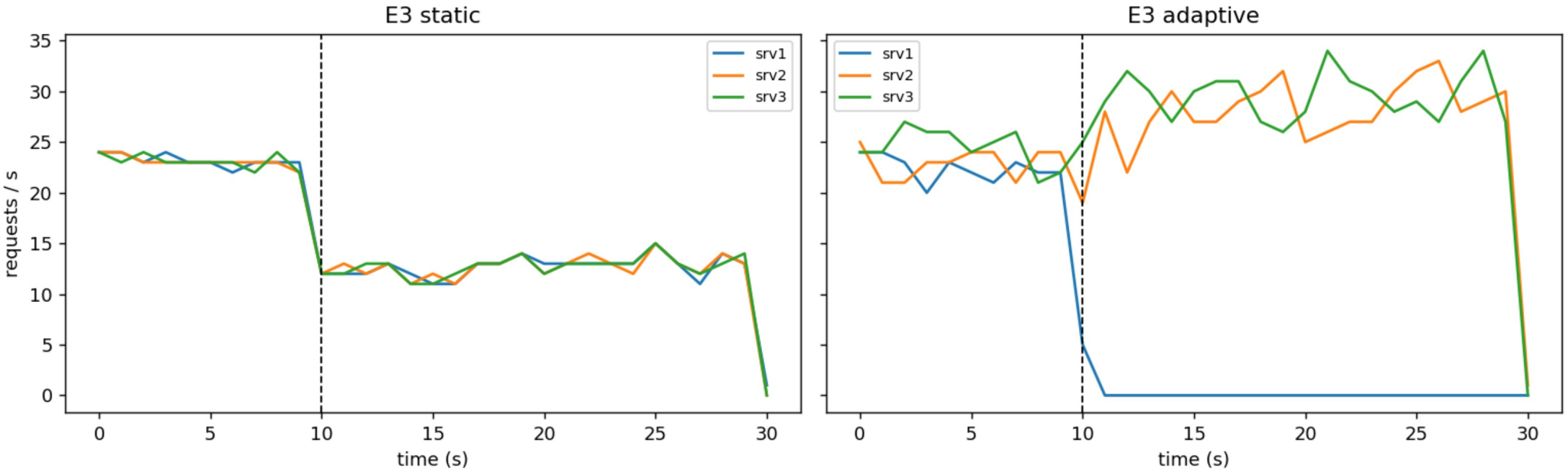
lower tail latency

At t = 10 s, h4 floods srv1 directly, bypassing the balancer. Only adaptive notices, from srv1's own load reports, and moves new flows away.

30 s per run · clients h1–h3 × 4 threads · h4 sends 12 threads of 500 ms requests to srv1

EXPERIMENT E3 · OVER TIME

srv1 is drained once it gets hot



The dashed line is when h4 starts flooding srv1. Left: static keeps srv1 at its share. Right: adaptive shifts srv1's share to srv2 and srv3.

EXPERIMENT E4 · SERVER FAILURE

A server dies: adaptive stops using it in 3 s

Failed requests (%), lower is better



Throughput (requests/s), higher is better



How long errors lasted after the kill (s)



RESULT

85%

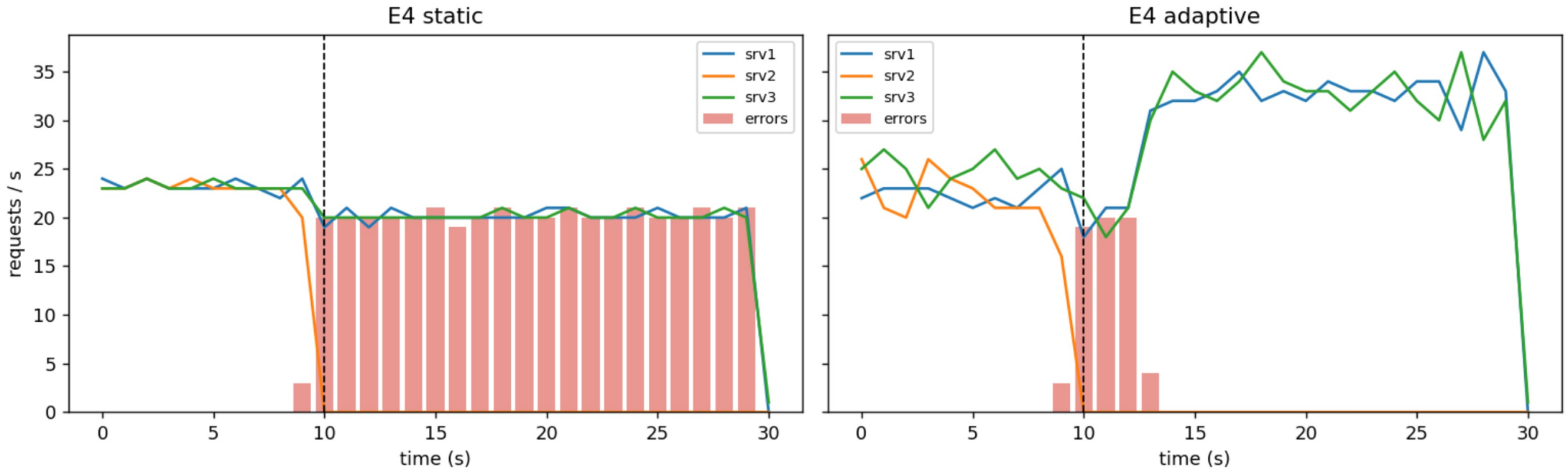
fewer failed requests

srv2 is killed at t = 10 s. Round-robin keeps sending it 1 in 3 new connections until the end. Adaptive marks it down after 3 s of silence and deletes its rules.

Repeated on the team's Mac VM (20 s runs): failed 21.0% → 5.9%, throughput 53.7 → 69.0 req/s

EXPERIMENT E4 · OVER TIME

Failures stop 3 seconds after the kill



Red bars are failed requests per second; the dashed line is the moment srv2 is killed. Right: after 3 s, srv1 and srv3 take over and throughput goes up.

EXPERIMENT E5 · LINK CONGESTION

A congested link: traffic moves to link B

Share of requests sent to srv3 on link B



p95 latency after t = 10 s (ms), lower is better



Throughput (requests/s), higher is better



RESULT

8%

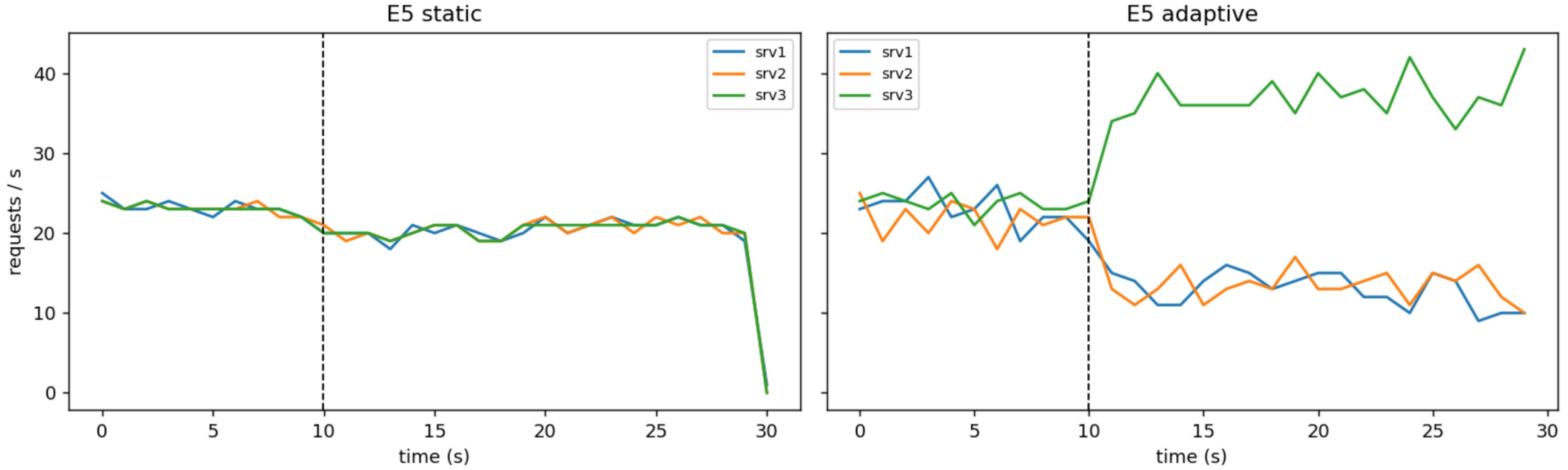
lower tail latency

h4 sends 9 Mbit/s of UDP across link A. Adaptive sees the link filling and moves traffic to srv3 on link B. The gain is small because responses are only 4 KB.

30 s per run · link A and link B capped at 10 Mbit/s · link load from OpenFlow port stats every 1 s

EXPERIMENT E5 · OVER TIME

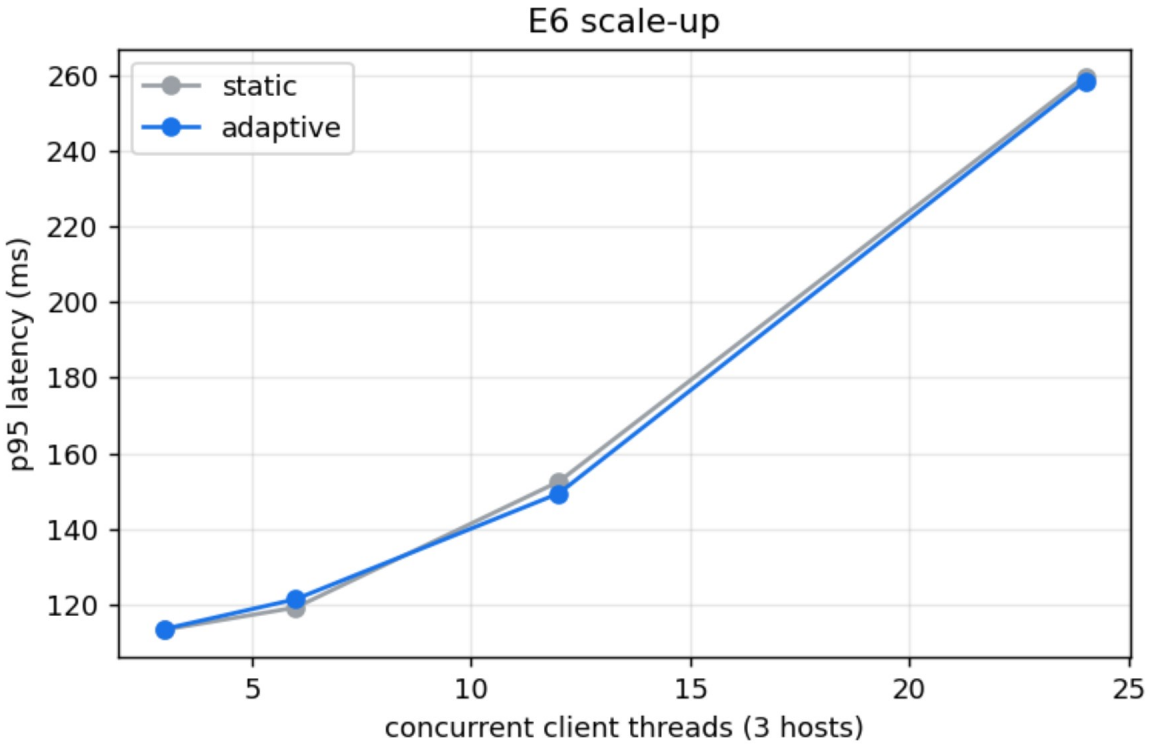
srv3 picks up traffic when link A fills



iperf starts at the dashed line and fills link A, which srv1 and srv2 share. Right: the srv3 line rises above the other two.

EXPERIMENT E6 · SCALE-UP

More clients: both modes scale the same way



Threads	Static p95	Adaptive p95
3	113 ms	114 ms
6	119 ms	121 ms
12	153 ms	149 ms
24	260 ms	258 ms

Throughput also matches: 103.7 vs 102.2 req/s at 24 threads. With equal servers there is nothing to fix, and the controller keeps up with about 100 decisions per second.

ALL SIX EXPERIMENTS

Adaptive wins where servers differ or fail

Experiment	Key metric	Static	Adaptive	Result
E1 balanced	p95 latency	152 ms	162 ms	Same
E2 weak server	Throughput	30.5 req/s	61.4 req/s	2× higher
E3 hot server	p95 after event	713 ms	212 ms	3.4× lower
E4 server killed	Failed requests	21.4%	3.3%	85% fewer
E5 busy link	p95 after event	184 ms	170 ms	8% lower
E6 24 threads	p95 latency	260 ms	258 ms	Same

30 s per run · 18 runs in total · full table in results/summary.md

AUTOMATED TESTS · ALL 8 PASS

What the tests check

Test	What it checks
test_static_is_round_robin_even_when_down	Round-robin keeps taking turns, even with a dead server
test_adaptive_prefers_least_loaded	The server with the fewest connections wins
test_pending_spreads_flows_between_reports	New flows spread out between two load reports
test_drain_and_resume_hysteresis	Draining at 90% of capacity, back below 70%
test_dead_server_skipped	A server silent for 3 s is marked down and skipped
test_fast_server_finishes_its_pending_flows	A fast server is not mistaken for a busy one
test_congested_link_shifts_to_other_path	A busy link makes servers on the other path win
test_server_and_client	Real TCP server and client on localhost; bad input rejected

Run: python3 tests/test_balancer.py and python3 tests/test_socket.py · passed on the team's Mac VM

Limits, and how we would fix them

One controller

If Ryu crashes, new connections can't be placed.

Fix: a backup controller.

3 s to detect a dead server

That window caused the 3.3% failures in E4.

Fix: active health checks from the controller.

Fixed topology

Ports and servers are written into the controller.

Fix: topology discovery; servers register themselves.

Trusts server reports

A server that reports wrong numbers gets wrong traffic.

Fix: also use response times the clients see.

TAKEAWAY

Adaptive balancing pays off when servers differ or fail.

2×
throughput with a weak server

3.4×
lower p95 with a hot server

85%
fewer failures when one dies

Questions?